UNIDAD N°3

Arquitecturas de Software

De la monolítica al microkernel: cómo se estructuran los sistemas y cuándo elegir cada enfoque

Taller de Lenguajes de Programación IV
Instituto Politécnico Formosa

Recorrido de la unidad

1 Introducción	2 Monolítica	3 Capas	4 Hexagonal	5 SOA
6 Microservicios	7 Event-Driven	8 Serverless	9 CQRS	10 Microkernel

El tema 10 (Microkernel) no está en el apunte original y se agrega como complemento.

¿Qué es una arquitectura de software?

Es la disciplina que define la estructura de un sistema: cómo se descompone en módulos o componentes y cómo estos se relacionan entre sí.

Las decisiones arquitectónicas tempranas afectan:

- **Escalabilidad**
- **Mantenibilidad**
- **Seguridad**
- **Flexibilidad**

Analogía: los planos de un edificio. Cambiar el color de una pared es barato; mover los cimientos cuando ya hay diez pisos construidos, no.

¿Por qué importa?

- Establece las bases sobre las que se desarrolla y evoluciona la aplicación
- Sin arquitectura adecuada: dificultad para escalar, tiempos de respuesta altos y mantenimiento costoso
- Una buena arquitectura resuelve el presente y permite adaptarse a cambios futuros

Dato de color: Los sistemas que crecen sin arquitectura tienen nombre propio en la literatura: "Big Ball of Mud" (gran bola de barro), término acuñado por Foote y Yoder en 1997.

Una evolución, no una competencia

Monolítica
Todo en una unidad

→

Por capas
Responsabilidades separadas

→

Hexagonal
Núcleo aislado del exterior

→

SOA
Servicios reutilizables

→

Microservicios
Servicios autónomos

Enfoques complementarios que se combinan con las anteriores:

Event-Driven	Serverless	CQRS	Microkernel

Dato de color: Ninguna arquitectura "reemplaza" a la anterior. Muchas empresas grandes conviven hoy con monolitos, SOA y microservicios en el mismo ecosistema.

Arquitectura Monolítica

Toda la aplicación vive en una única unidad de código que maneja interfaz de usuario, lógica de negocio y acceso a datos.

Ventajas

Desarrollo inicial fácil: un solo código base, rápido para proyectos chicos o medianos

Despliegue sencillo: una única unidad en el servidor

Desventajas

Escalabilidad limitada: todo está interconectado

Cambios inadvertidos: tocar una parte impacta en otras

Nuevas tecnologías: difícil modernizar solo una parte



Dato de color: Ejemplos típicos: apps creadas con Django o Laravel. El código se organiza en módulos, pero sigue siendo un único sistema desplegable.

Arquitectura de Capas

Divide la aplicación en segmentos con responsabilidad definida. Cada capa solo interactúa con la inmediata inferior, creando un flujo estructurado.

Capa de Presentación	Interfaz de usuario e interacción con el cliente
Capa de Lógica de Negocio	Las reglas que determinan cómo funciona el sistema
Capa de Acceso a Datos	Lectura y escritura en la base de datos

Beneficios

- Separación clara de responsabilidades; los desarrolladores se especializan en una capa
- Más fácil aislar y corregir errores

Limitaciones

- Puede volverse rígida: al crecer, la interacción entre capas se convierte en cuello de botella de rendimiento

Dato de color: Caso de estudio: app empresarial con Angular o React en presentación, Node.js o .NET en negocio, y PostgreSQL o MongoDB en datos.

Arquitectura Hexagonal

Busca desacoplar la lógica de negocio de las dependencias externas: bases de datos, APIs, servicios, interfaces de usuario. El núcleo no conoce el mundo exterior.

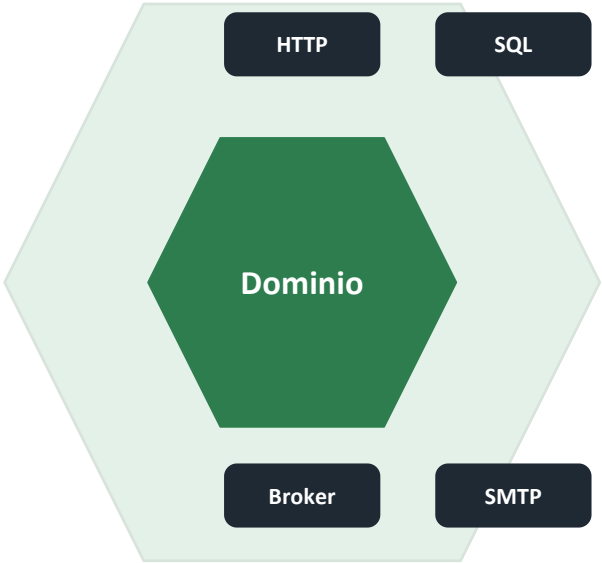
Puertos

- Interfaces que definen cómo la lógica de negocio interactúa con el exterior

Adaptadores

- Implementaciones de esas interfaces: conectan con bases de datos, APIs u otros sistemas

También conocida como Ports and Adapters



Puertos y adaptadores alrededor del núcleo

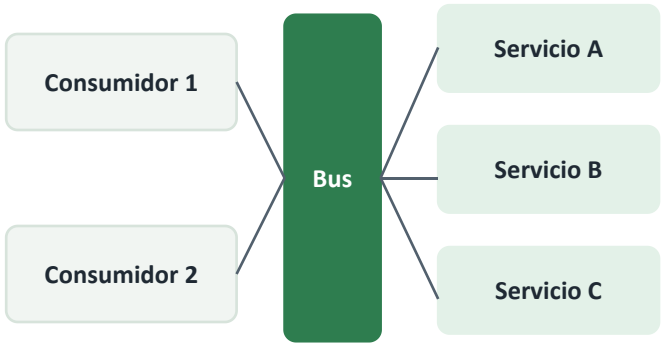
Dato de color: Aplica el Principio de Inversión de Dependencia (la D de SOLID). Fue propuesta por Alistair Cockburn en 2005. Ideal para sistemas de larga vida donde las dependencias cambian.

Arquitectura Basada en Servicios (SOA)

Los servicios son componentes que realizan una función específica e interactúan entre sí mediante interfaces de red. Se basa en la reutilización y en la flexibilidad para sistemas grandes.

Escenarios de aplicación

- Grandes organizaciones con muchos sistemas heredados que necesitan integrarse
- Aplicaciones escalables donde distintos sistemas reutilizan los mismos servicios
- Comunicación centralizada a través de un bus de servicios (ESB)



Los consumidores no hablan con cada servicio, hablan con el bus

Dato de color: SOA nació a fines de los 90 en el mundo empresarial, con protocolos como SOAP y XML. Muchos bancos y organismos públicos siguen operando sobre este modelo.

SOA vs Microservicios

Comparten la división del sistema en componentes autónomos, pero difieren en el foco.

Aspecto	SOA	Microservicios
Enfoque principal	Reutilización de servicios	Independencia total de cada componente
Comunicación	Bus de servicios centralizado (ESB)	Directa, ligera (HTTP/REST, mensajería)
Tamaño del servicio	Grande, de alcance empresarial	Pequeño, un dominio de negocio
Datos	Suelen compartir bases de datos	Cada servicio tiene su propia base
Despliegue	Coordinado entre servicios	Independiente por servicio
Escenario típico	Integrar sistemas heredados	Escalar productos digitales con equipos autónomos

Dato de color: Los microservicios se popularizaron alrededor de 2014 con el artículo de Martin Fowler y James Lewis; muchos los describen como "SOA bien hecha".

Microservicios

Evolución de SOA: la aplicación se divide en servicios pequeños, completamente autónomos y desplegables por separado. Cada uno atiende un dominio de negocio.



Principios

Independencia: desarrollar, desplegar y escalar sin afectar al resto

Modularidad: un servicio, un dominio de negocio

Desafíos

- Comunicación compleja entre servicios
- Duplicación de datos
- Transacciones distribuidas

Patrones y herramientas

API Gateway: punto de entrada único

Circuit Breaker: manejo resiliente de fallos

Docker y Kubernetes: contenedores y orquestación

Dato de color: Netflix migró a microservicios tras una falla de base de datos en 2008 que frenó los envíos de DVD por tres días. Amazon organiza sus equipos con la "regla de las dos pizzas".

Arquitectura Event-Driven

Las acciones dentro del sistema generan eventos que otros componentes capturan, procesan y a los que reaccionan de forma asincrónica. Ejemplo: un pedido cambia de estado y varios módulos reaccionan.



Ventaja: el productor no sabe ni le importa quién consume el evento. Desacoplamiento total.

Casos de éxito

Facebook: actualizaciones en tiempo real

Twitter y Uber: procesan grandes volúmenes de datos en tiempo real

Dato de color: Apache Kafka nació en LinkedIn en 2011 y hoy procesa billones de mensajes por día en empresas de todo el mundo. Su nombre homenajea al escritor Franz Kafka.

Arquitectura Serverless

Modelo de ejecución en la nube donde el proveedor gestiona la infraestructura. Las funciones se ejecutan en respuesta a eventos (una solicitud HTTP, un cambio en la base de datos) y se paga solo por el tiempo de ejecución.

Beneficios

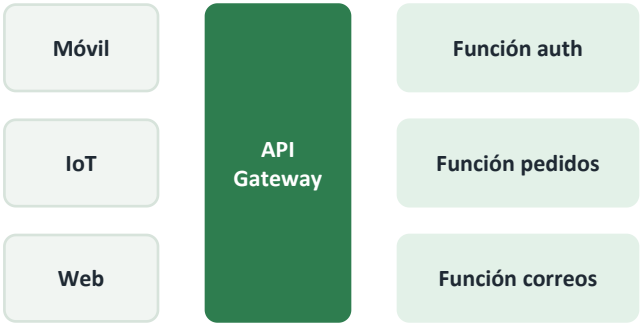
Escalabilidad automática: el proveedor ajusta recursos según demanda

Costos optimizados: se paga por uso real

Limitaciones

Latencia inicial: al activarse la función

No apto para cargas largas o constantes: está pensado para tareas cortas



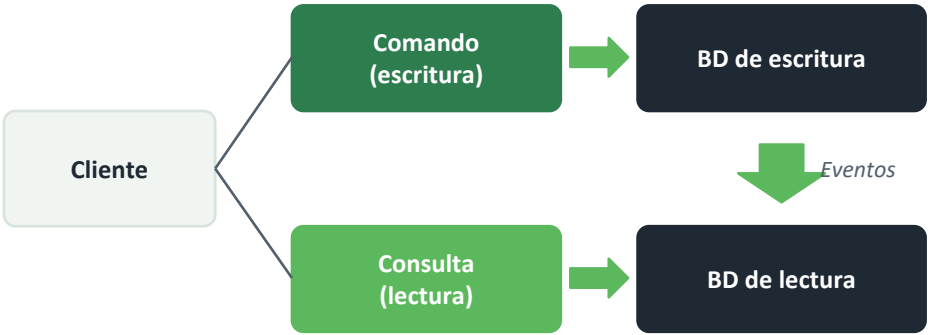
El desarrollador solo escribe las funciones

Servicios típicos: AWS Lambda, Azure Functions, Google Cloud Functions.

Dato de color: "Serverless" no significa que no haya servidores: los hay, pero son responsabilidad del proveedor. La latencia inicial se conoce como "cold start" (arranque en frío).

Arquitectura CQRS

Command Query Responsibility Segregation: separa las operaciones de lectura (consultas) de las de escritura (comandos), para optimizar cada una según sus necesidades.



Consistencia eventual: la lectura se actualiza poco después de la escritura

Por qué usarlo

Rendimiento: escrituras con transacciones y consistencia fuerte; lecturas optimizadas

Escalabilidad: lectura y escritura escalan por separado; ideal con muchas consultas y pocas actualizaciones

Event Sourcing: permite reconstruir el estado en cualquier momento a partir de los eventos; clave en sistemas financieros

Dato de color: El término CQRS fue acuñado por Greg Young alrededor de 2010. Un cajero automático es un buen ejemplo: consultar el saldo y hacer una extracción son caminos muy distintos.

Arquitectura Microkernel (Plug-in)

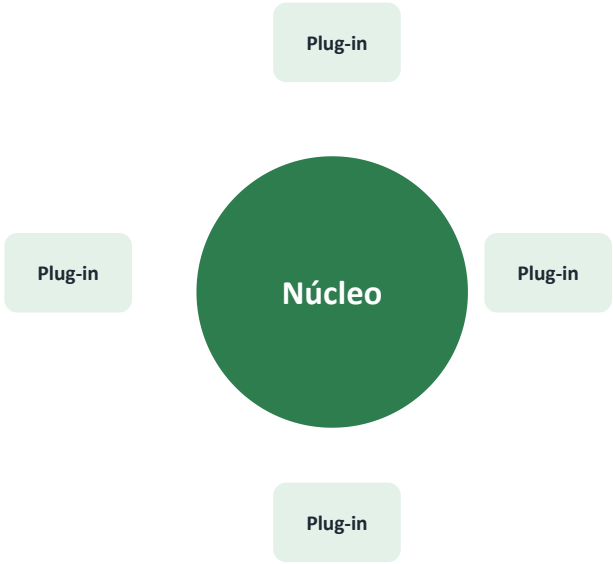
Un núcleo mínimo con la funcionalidad esencial, y un conjunto de módulos o plug-ins que agregan capacidades sin modificar el núcleo. Los plug-ins se comunican con el núcleo a través de contratos definidos.

Ventajas

- Extensible sin tocar el núcleo
- Plug-ins aislados: fallan sin tumbar el sistema
- Funciones opcionales según cliente

Desventajas

- Definir bien el contrato del núcleo es difícil
- Gestión de versiones entre plug-ins
- No escala tan bien como microservicios



Extensiones alrededor de un núcleo estable

Dato de color: Ejemplos: VS Code y Eclipse (extensiones), WordPress (plugins), Jenkins, navegadores web. En sistemas operativos, MINIX y QNX aplican esta idea a nivel de kernel.

¿Cuándo usar cada una?

Arquitectura	Úsala cuando...	Evítala cuando...
Monolítica	El proyecto es chico o mediano y el equipo es reducido	Necesitás escalar partes por separado
Capas	Querés responsabilidades claras y equipos especializados	El rendimiento entre capas es crítico
Hexagonal	La lógica de negocio debe probarse aislada y vivir muchos años	Es un prototipo descartable
SOA	Hay muchos sistemas heredados por integrar	Buscás autonomía total por equipo
Microservicios	Equipos autónomos y escalado por dominio	No tenés madurez en DevOps y contenedores
Event-Driven	Muchas partes deben reaccionar a un mismo suceso	Necesitás respuestas síncronas simples
Serverless	Cargas variables y tareas cortas	Procesos largos o latencia muy baja
CQRS	Muchas más lecturas que escrituras	Un CRUD simple alcanza
Microkernel	El producto debe extenderse con módulos opcionales	No hay un núcleo estable que aislar

Datos de color para recordar

Ley de Conway (1967)

Los sistemas terminan reflejando la estructura de comunicación de la organización que los construye.

Regla de las dos pizzas

En Amazon, un equipo no debe ser más grande de lo que pueden alimentar dos pizzas. Cada equipo, sus servicios.

Cold start

Una función serverless dormida puede tardar cientos de milisegundos en despertar. Por eso no sirve para todo.

Big Ball of Mud

La "arquitectura" más común del mundo real: la que no se diseñó. Foote y Yoder, 1997.

Monolito primero

Martin Fowler recomienda empezar con un monolito bien modularizado y extraer microservicios solo cuando duele.

Kafka en LinkedIn

El broker de eventos más usado del mundo nació para procesar la actividad de una red social profesional.

Para cerrar

- La arquitectura es el conjunto de decisiones que después cuesta cambiar
- No hay una arquitectura mejor: hay una más adecuada al problema, al equipo y al momento
- Las modernas (Event-Driven, Serverless, CQRS, Microkernel) se combinan con las clásicas, no las reemplazan
- Empezar simple y evolucionar suele ganarle a empezar complejo

Pregunta para debatir: ¿qué arquitectura tiene el proyecto que están desarrollando hoy? ¿Y cuál debería tener en dos años?